Consolidated Defense Knowledge Base

ChainCert

Blockchain-Based Credential Issuance and Verification Platform

March 25, 2026 — CAP-2459-IT

Acknowledged Limitations & Weaknesses

The following are known limitations, trade-offs, and areas for improvement that the team acknowledges. These are not oversights but conscious design decisions within the capstone scope, and each has a prepared defense rationale.

Governance & Trust

- Administrator remains a single point of failure for governance — can approve fake issuers, but every action is immutably logged on blockchain (detection, not prevention).
- Governance is centralized by design — appropriate for a platform model, but distributed governance is architecturally supported via Fabric's configtx.yaml policy mechanism.
- Garbage-in risk — blockchain makes data immutable but does not guarantee legitimacy of data entering the system. Exists across all three deployment designs.
- Onboarding legitimacy depends on admin validation quality — document review is a human process.

Security

- JWT secret has a hardcoded fallback ('dev_secret_key') if .env is missing. Production: remove fallback.
- CouchDB credentials hardcoded as admin:adminpw across all files. Production: use environment variables.
- JWT tokens stored in localStorage (XSS-vulnerable). Production: migrate to httpOnly cookies.
- No rate limiting on any endpoint. Production: add express-rate-limit.
- All traffic over HTTP, not HTTPS. Production: add TLS termination.
- Disabled users' JWT tokens remain valid up to 24 hours (no token blacklist).
- Shared Fabric admin identity — no per-admin cryptographic attribution. Individual accountability via logged metadata.

Architecture

- Single orderer node — single point of failure. Production: Raft consensus cluster with 3-5 orderers.
- All infrastructure on one machine — 3 ledger copies on same server don't provide genuine infrastructure redundancy.
- Mutable CouchDB read layer — mitigated by blockchain verification + integrity check, but UI could display tampered data between integrity checks.

- One-directional integrity check (CouchDB → blockchain only) — orphaned blockchain records with missing CouchDB copies would not be detected.

Data & Operations

- Dual-write pattern with no multi-database transaction support — partial failures handled by warnings, not automatic rollback.
- Credential expiry has a timing gap — credentials remain 'valid' on blockchain until the cron job or admin manual trigger runs. No real-time expiry check in VerifyCredential chaincode.
- No notification on credential expiry — holder's credential silently becomes expired.
- Duplicate prevention relies on CouchDB, not blockchain-level check — partial write failure could allow duplicate issuance.
- Credential IDs use Date.now() + Math.random() — not cryptographically secure, but ID is not a security token.
- No formal throughput benchmarking performed.

Features Not Implemented

- Private Data Collections (selective visibility between orgs on same channel).
- Distributed channel governance (multi-org signing for config updates).
- Automated node provisioning (requires tools like Hyperledger Bevel).
- Blockchain-to-CouchDB reverse integrity scan.
- Cron-based CouchDB-blockchain synchronization.

Table of Contents

Acknowledged Limitations & Weaknesses

1. Governance / Current Model / Consortium Possibility

2. Problems Solved vs Problems Not Fully Solved

3. Verification Module / Trust Model

4. System Architecture (with diagrams)

5. Hyperledger Fabric / Design Choices

6. Issuer Onboarding / Operational Reality

7. Security / Admin / Identity Model

8. Feature Inventory

9. Data Design / CouchDB

10. Implementation / Code Flow

11. Deployment / Network Notes

12. Miscellaneous Design Decisions

13. Hostile Panelist Questions — High Risk

14. Hostile Panelist Questions — Design-Level Attacks

15. Hostile Panelist Questions — Architectural Attacks

Final Practical Reminders

1. Governance / Current Model / Consortium Possibility

> ❓ **What is the correct governance and deployment model of ChainCert?**

Tropang Exp:

Chaincert handles everything, para wala ng iisipin nga issuers/user but to just use the system. May disadvantages to it, pero intentional trade off/decision siya kasi para mas masimplify process ng pag gamit for issuers/users. Architecturally, the setup where issuer deploys their node + has a say on who gets in is still possible may konting icoconfigure lang, sadyang itong current setup where we handle everything yung ginawa muna natin.

Summary:

ChainCert is a credential issuance and verification platform with a SaaS-style model where ChainCert admin handles infrastructure and issuers use the platform as clients. ChainCert handles: node deployment, MSP identities, onboarding approval, and blockchain integration. Issuers do not own or manage their own nodes.

Technical Detail:

The current implementation follows a centrally managed permissioned blockchain model. The network is permissioned, only approved issuers can participate, and ChainCert acts as platform operator, onboarding authority, and infrastructure manager. The proposal requires permissioned access, authorized institutions, controlled onboarding, and immutable verifiable credentials. It does NOT require that every issuer owns its own node, participates in blockchain-level voting, or that every new org must be approved by all existing orgs.

> ✅ **Defense Answer**
>
> *"Our current model is intentionally platform-managed. Since ChainCert functions as a credential issuance and verification service, it is appropriate that infrastructure, onboarding, and network management are handled centrally by ChainCert. This satisfies the specifications of a permissioned blockchain system while keeping onboarding efficient and practical for participating institutions."*

> ❓ **What is the current model vs the future optional consortium model?**

Tropang Exp:

- **Current Model:** Chaincert handles everything, including node deployment and who gets to be an issuer
- **Future Design 2:** Issuer deploys, owns, and handles their own node via deployment kit given by chaincert. Why? Ayaw magtiwala ng org satin na tayo maghandle ng data nila, gusto nila may independent access and copy rin sila ng ledger by means of handling their own node. NOTE, decision on who gets to be an issuer is still depedent on chaincert
- **Future Design 3:** Basically design 2 but all orgs have a say on who gets to be an issuer, not just chaincert handling everything

Summary:

Current model: ChainCert alone decides who joins the blockchain network and manages/deploys nodes for the issuer (centralized). Future optional model: ChainCert AND existing issuers have a say on who joins (decentralized), requiring issuers to own their node. ChainCert gives them a deployment kit and they deploy their own peer, CA, and CouchDB. The "existing org approval" mechanism is Fabric channel config updates that require cryptographic signatures from multiple existing org admins.

Technical Detail:

In the consortium model, adding a new org requires a channel configuration update signed by a majority of existing org admins. The commands involved are the same ones used in node creation (configtxlator, peer channel signconfigtx, peer channel update) — except currently ChainCert signs everything because it controls all admin certs. In a consortium, those signing steps would be distributed to different organizations running their own infrastructure.

> ✅ **Defense Answer**
> *"The current implementation uses centralized onboarding because it best fits the platform-based nature of ChainCert. However, Hyperledger Fabric supports future expansion into a more distributed governance model if clients later require network-level consensus among participating organizations."*

> ❓ **What are the possible deployment architectures and why was the current one chosen?**

Tropang Exp:

Same answer in tropang exp of " ❓ What is the current model vs the future optional consortium model?". Kaya chaincert handles everything ang implemented right now kasi:

- Simplifies onboarding process, normally ayaw ng issuer ng added responsibility of deciding who gets in, pake lang non makapagissue

- Ito base purpose ng proposal natin, makapagissue and verify securely, nothing explicitly mentioned about issuers are required to own their node and participate in deciding which issuers gets in
- Sa UAT, technically ang hirap non, for example we did design 2, meaning kada user sesendan natin ng deployment kit tas sila pa magdedeploy, jusko pake nila don, simply put para wala na silang icoconfigure at gagamit nalang sila ng system natin for UAT

Summary:

Three designs are architecturally possible:

Design 1 (Current — Platform): ChainCert manages all infrastructure and governance. Issuers use the platform as a service.

Design 2 (Hybrid): Issuers deploy their own nodes for data sovereignty, but ChainCert retains sole governance authority. configtx.yaml Admins policy stays OR(Org1MSP.admin). Node ownership separated from governance.

Design 3 (Full Consortium): Issuers deploy their own nodes AND participate in governance. configtx.yaml Admins policy changes to MAJORITY(all member MSPs). New member admission requires cryptographic signatures from majority of existing members.

Design 1 was chosen because: (a) the core value proposition (blockchain-backed credential issuance, verification, auditability) functions identically regardless of who hosts infrastructure, (b) Design 2 introduces infrastructure distribution complexity outside capstone scope, (c) the prototype operates on a single machine with no actual external institutions so Design 2 provides zero real data sovereignty, (d) UAT testers would need to deploy their own Fabric nodes before testing — making UAT impractical, and (e) the system demonstrates extensibility through documented Fabric policy mechanisms without implementing them.

Pros/cons per design:

Design 1: Fastest onboarding, simplest operations, lowest barrier, cheapest. But single point of compromise, no real data sovereignty, scalability ceiling.

Design 2: Genuine data sovereignty, genuine infrastructure redundancy, satisfies government requirements, simple governance. But higher onboarding complexity, requires deployment kit and support.

Design 3: Maximum decentralization, strongest trust model. But slowest onboarding, governance complexity, highest cost, only makes sense for equal-stake institutional consortiums.

> ✅ **Defense Answer**

> *"We identified three deployment architectures — fully managed, hybrid with issuer-owned nodes, and full consortium. We chose the fully managed platform model because it is sufficient to demonstrate the system's core contribution. The transition to hybrid or consortium architectures is supported by Fabric's policy configuration and documented as a future path when institutional demand for data sovereignty or distributed governance arises."*

> ❓ **What are the detailed technical processes for each deployment architecture?**

Tropang Exp:

- **Design 1: Chaincert handles everything (Current)**
  - Follow installation manual lang
- **Design 2: Issuer owns their node, but Chaincert decision Maker in who gets to be an Issuer**
  - Give deployment kit kay issuer
  - Deploy sila
  - After doing so, may certain files and details silang ibibigay satin para malink natin yon sa account nila/maconfigure natin what is needed.
- **Design 3: Design 2 pero all orgs have a say on who gets to be an issuer**
  - Same steps with design 2
  - Only difference being is may iibahin tayo sa config.tx file something para maconfigure na dapat lahat may say on who gets in

Summary:

Design 1 process (current): ChainCert performs all phases on their server — deploy CA container (compose-ca-dlsu.yaml, port 12054), register and enroll identities (registerEnrollFORDLSU.sh — fabric-ca-client register/enroll), deploy peer container (compose-dlsu.yaml, port 12051, CORE_PEER_LOCALMSPID=DLSUMSP), deploy CouchDB (compose-couch-dlsu.yaml, port 9084), update channel config (configtx-WITHDLSU.yaml adds DLSU org definition), join peer to channel, install and approve chaincode, import wallet identity (setupWallet_DLSU.js), create CouchDB databases, register couchdb_connections mapping, click approve in UI.

Subordinate process (simpler): Register identity in root CA (fabric-ca-client register with --id.type client), enroll MSP, import wallet, create CouchDB container + databases, register couchdb_connections, approve. No channel config update needed.

Design 2 process: Issuer performs infrastructure phases on their own server using deployment kit from ChainCert. Issuer sends back: MSP definition, peer endpoint, peer TLS certificate, admin certificate. ChainCert adds org to configtx.yaml, generates and signs channel config update, submits. Key backend change: connection profile peer URL changes from localhost to issuer-server, gateway discovery asLocalhost changes from true to false.

Design 3 process: Infrastructure identical to Design 2. Only difference: channel config update signing. Signed envelope must be passed to each existing member org, each runs peer channel signconfigtx, once majority signs the envelope is submitted. Requires coordination mechanism and governance workflow.

The configtx.yaml Admins policy is the single configuration point that determines whether the setup is Design 1, 2, or 3.

> ✅ **Defense Answer**
>
> *"The technical deployment process scales from fully managed to hybrid to consortium. The key configuration point is the Admins policy in configtx.yaml — changing it from OR(Org1MSP.admin) to MAJORITY(all member MSPs) transitions governance from centralized to distributed without changing application code, chaincode, or frontend."*

❓ Can issuers own their nodes while ChainCert retains full governance control?

Summary:

Yes. Node ownership and governance authority are separate configurations in Hyperledger Fabric. The channel governance policy (configtx.yaml Admins rule) controls who must sign for channel configuration changes — this is independent of who operates peer nodes. An issuer can own and operate their own peer for data sovereignty and independent verification while ChainCert retains sole onboarding authority.

This enables a hybrid deployment model: Tier 1 (Managed) — small issuers use ChainCert-managed infrastructure. Tier 2 (Self-hosted) — large institutions deploy their own peer, CA, and CouchDB, join chaincert-channel, maintain independent ledger copy, but have zero governance voting rights. Both tiers coexist on the same channel.

Owning a node does NOT obligate an organization to sign governance decisions. The Admins policy in configtx.yaml must be explicitly edited to grant governance rights. No edit = no governance rights regardless of how many nodes they run.

> ✅ **Defense Answer**
>
> *"Fabric allows us to separate node ownership from governance authority. An institution can deploy its own peer node for data sovereignty and independent ledger verification, while ChainCert retains sole control over onboarding and network governance. This hybrid model serves both small organizations that prefer managed infrastructure and large institutions that require sovereign control — all on the same channel, all verifiable through the same interface."*

❓ Why is the current setup actually the best fit for the system?

Summary:

Because the system is a credential platform, not a peer-governed alliance of equal institutions. Issuers join to issue and verify credentials, not to govern other issuers, vote on membership, or co-manage network policy. Forcing multi-org governance now would: slow onboarding, give issuers extra responsibility they don't need, explode governance complexity, and make the system less practical.

Technical Detail:

The use case is much closer to admin-reviewed accreditation and managed onboarding than to equal-stake consortium governance. This is analogous to CHED/accreditation: schools apply, authority reviews, schools do not vote on other schools. Or like Credly/SaaS: platform owner controls infrastructure, institutions use the service.

> ✅ **Defense Answer**
>
> *"The current setup is the best fit because ChainCert is designed as a platform-based credential service, not as a consortium where participating institutions are expected to govern each other. Centralized onboarding is therefore more efficient, realistic, and aligned with the actual responsibilities of issuers in the system."*

❓ When would blockchain-level multi-org approval become useful?

Summary:

Only when ChainCert evolves beyond a platform and becomes a shared network of equal institutions. Examples: national academic consortium, cross-border academic recognition network, regulator + university + licensing-body networks.

Real-world examples where participating organizations have a say in who joins:

Blockchain: R3 Corda Network (banks approved by existing members), TradeLens (Maersk + IBM — carriers/customs required endorsement), Sovrin Network (SSI — steward orgs approved by governance board), We:Trade (European trade finance consortium on Fabric).

Non-blockchain parallels: VISA/Mastercard (new issuing banks require sponsorship from existing members), SWIFT (new banks must be approved), PAASCU (Philippine accreditation — peer teams from accredited institutions review applicants), The European Union (new member states require unanimous approval under Copenhagen criteria).

The common principle: when one member's actions affect the credibility of the entire network, existing members must have a say in who joins.

> ✅ **Defense Answer**
>
> *"Blockchain-level multi-organization approval becomes useful only when the system evolves into a true consortium where participating institutions share governance responsibility. In our current platform model, this would be unnecessary overhead rather than a required capability."*

> ❓ **In a multi-org consortium, what basis do existing organizations use to approve a new one?**

Summary:

Two layers of approval criteria:

Blockchain layer (technical): Existing organizations review the new org's MSP configuration — CA certificates, root certificates, organizational unit definitions. They verify the new org has properly configured infrastructure (peer node, CA, TLS certificates) and that their MSP is cryptographically valid. The signing mechanism is Fabric's channel configuration update requiring cryptographic signatures from a majority of existing org admins.

Application layer (legitimacy): Same document-based validation ChainCert already does — CHED recognition, SEC registration, board resolutions, etc. The difference is that this review is distributed across multiple member organizations instead of one admin.

So the basis is: technical readiness (MSP, peer, CA are valid) + institutional legitimacy (same document validation, but distributed across multiple reviewers).

> ✅ **Defense Answer**
>
> *"In a consortium model, approval happens at two layers. At the blockchain layer, existing organizations verify the new org's MSP configuration and infrastructure readiness through*

> *cryptographic signing. At the application layer, the same document-based validation we currently use would be distributed across multiple member organizations instead of a single admin."*

❓ In a consortium setup, can participating organizations create their own channels, or only ChainCert admin?

Summary:

In Hyperledger Fabric, channel creation requires a channel configuration transaction signed by organizations that have the authority to do so, as defined in the orderer's system channel configuration. In the current ChainCert setup, only ChainCert admin can create channels because ChainCert controls the orderer and all admin certificates. In a full consortium setup, the policy could be configured to allow member organizations to propose and create new channels — but even then, proposals require approval (signatures) from enough existing orgs based on the policy. It is never a unilateral action.

> ✅ **Defense Answer**
> *"Channel creation authority is controlled by the orderer's consortium policy. In our current setup, only ChainCert admin can create channels. In a consortium model, this authority could be distributed to member organizations — but channel creation always requires approval signatures from existing members, it is never unilateral."*

❓ Why not implement the multi-channel design with selective hashing in the current setup?

Summary:

Three reasons:

First, it solves a problem we don't have yet. Multi-channel with selective hashing protects data privacy between independent organizations that don't trust each other. In our current setup, ChainCert controls all infrastructure — there are no independent organizations with competing privacy interests.

Second, it requires multi-org infrastructure. Private channels only make sense when each institution operates their own peer node, because the privacy guarantee is that only peers joined to a channel can see that channel's data.

Third, scope and complexity. Multi-channel architectures require separate chaincode deployment per channel, cross-channel transaction coordination, hashing logic, consent mechanisms, and channel-aware integrity checks — significant engineering that exceeds capstone scope.

✅ **Defense Answer**

"Multi-channel with selective hashing solves a data privacy problem between independent organizations. In our current platform model, ChainCert controls all infrastructure, so there are no competing privacy interests — adding private channels would increase complexity without providing actual privacy benefit. This architecture becomes necessary only when institutions operate their own peer nodes and need to protect their data from other consortium members."

❓ What would motivate ChainCert to move from platform model to consortium?

Summary:

ChainCert would not voluntarily move to consortium — the platform model is simpler, cheaper, and gives ChainCert full control. The shift would be driven by external pressure from clients:

(1) Institutional trust demands: Large institutions may refuse to store official credential records on infrastructure they don't control.

(2) Regulatory requirements: Government agencies often have data sovereignty mandates.

(3) Security at scale: With all peers on one machine, one breach compromises all ledger copies simultaneously.

(4) Cross-institutional recognition: Institutions will demand their own nodes and verification capability.

(5) Trust asymmetry: As ChainCert handles more valuable credentials, clients will demand distributed controls.

The motivation comes from the clients, not from ChainCert.

✅ **Defense Answer**

"The consortium model becomes necessary when the platform's clients — particularly large institutions and government agencies — require sovereign control over their infrastructure and data. ChainCert's current platform model is appropriate for our scope, but the architecture supports that transition when client demand requires it."

❓ What would a production consortium architecture look like specifically?

Summary:

A multi-channel design with selective hashing. Each institution gets a private channel storing full credential records, plus a shared general channel storing privacy-preserving versions (hashed credential IDs, redacted holder details, only non-sensitive metadata in plaintext). Verification queries the general channel to confirm validity without exposing full details. Full details require holder consent to query the private channel. Each institution would operate its own CA, peer node(s), and MSP — onboarded through governance where existing members vote to approve new org configurations.

> ✅ **Defense Answer**
>
> *"For a production consortium, we recommend a dual-channel architecture: each institution operates a private channel for full credential records, while a shared general channel stores privacy-preserving versions with hashed identifiers. This enables cross-institutional verification without exposing sensitive data. The current single-channel architecture was chosen because it demonstrates the core value proposition without the operational complexity of multi-org infrastructure, which exceeds capstone scope."*

2. Problems Solved vs Problems Not Fully Solved

❓ What problems are actually solved by the system?

Summary:

Solved in current implementation (Design 1): credential tampering (on-chain records are immutable and auditable), slow/unreliable verification (verifiers validate through the system rather than manually contacting issuers), lack of auditability (issuance, revocation, verification, and governance events are all logged and traceable on blockchain), credential lifecycle management (issuance, expiration, revocation all tracked on-chain).

Partially solved: data-level decentralization — credential records are on an immutable ledger with independent verification logic, but all peer nodes are on the same machine, so infrastructure-level decentralization is not achieved in the current deployment. The data trust model is decentralized; the infrastructure hosting it is not yet.

Not solved by choice: governance decentralization — onboarding is centralized by design because it fits the platform model. This is not a limitation of the architecture but a deliberate design choice. The system architecture supports distributed governance (Design 3) through Fabric's configtx.yaml policy mechanism, but this was not implemented because it would add governance complexity without improving the core credential integrity and verification capabilities.

Important distinction: saying "the architecture supports Design 3" is not the same as saying "we solved governance decentralization." The current system has centralized governance. The architecture can evolve to support distributed governance when client demand requires it.

✅ **Defense Answer**

"Our system directly addresses credential tampering, slow verification, lack of auditability, and credential lifecycle management through blockchain-backed issuance and verification. Data-level trust is decentralized through blockchain immutability, though infrastructure-level decentralization requires issuer-owned nodes which are outside the current deployment scope. Governance remains centralized by design — the architecture supports a transition to distributed governance through Fabric's policy configuration, but this was not implemented because it adds complexity without improving the core credential integrity capabilities."

❓ Did the system solve centralization?

Summary:

There are three levels of centralization to consider:

(1) Centralized data trust — traditional systems rely on one database that can be silently altered. Solved. Blockchain immutability means credential records cannot be modified without detection. The integrity check validates that off-chain data matches on-chain records. Verification goes through blockchain-backed logic, not just a database lookup.

(2) Centralized infrastructure — all peers are on one machine. Not solved in Design 1. The "3 independent ledger copies" guarantee is weakened because all copies are on the same server. True infrastructure decentralization requires at least some peers on independently operated servers (Design 2).

(3) Centralized governance — one admin controls onboarding. Not solved in Design 1 or 2. Distributed governance requires Design 3.

The proposal's primary concern was centralized data trust (tamperable records, dependence on one system, inability to verify independently). This is solved.

> ✅ **Defense Answer**
>
> *"We addressed the centralization problem most relevant to the proposal — centralized trust over credential records. Blockchain immutability and independent verification logic ensure that no single party can silently alter credential data. Infrastructure centralization (all peers on one server) and governance centralization (admin controls onboarding) remain as acknowledged trade-offs. Infrastructure decentralization is achievable through issuer-owned nodes; governance decentralization is achievable through Fabric's policy configuration. Both are documented future extensions."*

❓ What kind of centralization was meant in the proposal?

Summary:

The main centralization problem in the proposal was centralized trust over records, not "all governance must be decentralized." The proposal's concern refers primarily to: tamperable centralized records, dependence on one system/database, and inability for third parties to verify independently.

> ✅ **Defense Answer**
>
> *"The centralization problem identified in the proposal primarily referred to centralized control over credential records and the lack of independent verification. Our solution addresses this directly*

> *through blockchain immutability and auditability, while the centralized governance that remains is an accepted trade-off of the chosen permissioned model."*

❓ Is admin still a single point of failure?

Summary:

Yes — in governance. The admin controls issuer admission, account status changes, and platform configuration. This is a deliberate choice: we prioritized onboarding simplicity because the system is a credential platform, not a peer-governed consortium.

Critical distinction from traditional systems: admin controls who joins, but cannot silently alter credential history. Every governance action (approval, rejection, enable, disable) is immutably recorded on blockchain. A compromised admin can approve fake issuers, but cannot hide that approval. This is accountability, not prevention.

Can this be solved? Design 3 distributes governance across multiple orgs — no single admin controls admission. However, Design 3 does not eliminate single points of failure; it redistributes them. New risks emerge: majority coalition collusion, signing org going offline blocking admissions, diffusion of responsibility.

> ✅ **Defense Answer**
>
> *"The administrator remains a single point of failure for governance — this is a deliberate trade-off for onboarding simplicity. The critical improvement is that the admin can no longer act without accountability. Every governance decision is immutably logged on blockchain. A distributed governance model is architecturally supported through Fabric's policy configuration but would introduce coordination complexity and new failure modes that are not warranted for the current platform scope."*

❓ What about "Garbage In, Immutable Garbage Out"?

Summary:

This is a fundamental limitation that exists across ALL three designs, not just Design 1. Garbage-in is not solved by distributing the approval decision across more people. In Design 3, a majority of orgs could still approve a fake issuer through collusion, negligence, or rubber-stamping. More signers does not mean better validation — it can actually worsen the problem through diffusion of responsibility.

What actually mitigates garbage-in is the quality of the validation process: stronger document requirements, third-party verification, cross-referencing with external databases. These are application-layer improvements independent of which deployment design is used.

What blockchain provides regardless of design: if garbage does enter, it is permanently recorded, detectable, traceable, and correctable. In a traditional database system, a fraudulent approval can be silently deleted. In ChainCert, it cannot.

> ✅ **Defense Answer**
>
> *"Garbage-in is a fundamental limitation of any system that relies on human judgment for admission, regardless of how many approvers are involved. Distributed governance does not eliminate this risk — it distributes it. What blockchain provides is that once actions occur, they are permanently recorded and cannot be hidden. Fraudulent activity is detectable, traceable, and correctable through credential revocation and audit logs. This is a significant improvement over traditional systems where such activity can be erased entirely."*

3. Verification Module / Trust Model

❓ What are the different ways a verifier can verify credentials?

Summary:

A verifier can verify through: (1) Exported PDFs sent by the user (may contain links to profile or specific credentials), (2) User-shared links (public profile or badge preview pages), (3) Verifier Dashboard where verification is done through manual credential ID input or direct link/QR flow.

The Verifier Dashboard matters because all other paths are presentation-layer outputs fetched from CouchDB or editable PDFs. The dashboard is the official trust layer that checks against blockchain-backed verification logic.

Technical Detail:

The system has multiple viewing/sharing paths but not all are equally trustworthy. PDFs, profile preview links, and credential/badge preview pages are convenience paths — fast, user-friendly, but fetched from off-chain data. The Verifier Dashboard is the trust/validation path that performs blockchain-backed verification.

✅ Defense Answer

"Our system supports multiple verification entry points for convenience, such as PDFs, profile links, and credential preview pages. However, those are still presentation-layer outputs and may rely on off-chain data. The Verifier Dashboard exists as the authoritative validation layer, allowing verifiers to confirm credential authenticity and status through blockchain-backed verification rather than relying solely on what was shared by the user."

❓ Why use CouchDB for display and blockchain for verification?

Summary:

CouchDB = performance. Blockchain = trust. They are complementary, not competing. Blockchain is not ideal for high-frequency UI reads (dashboards, profile pages, wallet views, PDF generation). If everything were fetched directly from blockchain, UI would be slower, queries would be less flexible, and the system would become less practical.

✅ Defense Answer

> *"CouchDB is used for efficient display and querying, while blockchain remains the source of truth for validation and auditability. This architecture provides both usability and integrity."*

❓ If CouchDB can be tampered with, does that make blockchain useless?

Summary:

No. CouchDB should not be trusted for critical validation. Blockchain is still the immutable reference point. If CouchDB is modified, UI may show incorrect info, but if verification is done through the blockchain-backed path, mismatches become detectable. The system already includes an admin-triggered integrity check that performs field-by-field comparison (credentialName, credentialType, holderID, status, issuedDate, expiryDate, description, issuerId) between CouchDB and blockchain using the Get_CredentialTemplateOrIssuedCredential chaincode function.

> ✅ **Defense Answer**
> *"CouchDB is a mutable performance layer, but the blockchain remains the immutable source of truth. Off-chain tampering is theoretically possible but detectable through blockchain-backed verification and the built-in integrity check that compares CouchDB records against blockchain field by field."*

❓ Why not use a cron job to continuously sync CouchDB with blockchain?

Summary:

Synchronization is a consistency mechanism, not a real-time security guarantee. A cron-based sync helps with consistency and reconciliation but does not prevent a verifier from seeing tampered data between sync intervals. The better model is: CouchDB for display, blockchain verification on demand when trust matters.

> ✅ **Defense Answer**
> *"Periodic synchronization may improve consistency, but it does not provide real-time trust guarantees. That is why final validation should still rely on blockchain-based verification rather than delayed background reconciliation."*

❓ Is credential ID enough for verification? What about QR codes or direct links?

Summary:

Yes, credential ID is technically sufficient. QR codes and direct links are just better UX paths to the same verification result. All resolve to the same credential ID, the same verification logic, and the same blockchain-backed result.

> ✅ **Defense Answer**
>
> *"Credential ID is technically sufficient for verification because it uniquely identifies the record to be checked. QR codes and direct verification links simply improve usability by passing that same identifier automatically into the verification flow."*

4. System Architecture

This section documents the actual running architecture of ChainCert as deployed.

4.1 Blockchain Network Components

The Fabric network has 1 orderer, 3 peers (one per org), 4 CAs, and 3 peer state CouchDBs, all on a single channel called chaincert-channel.

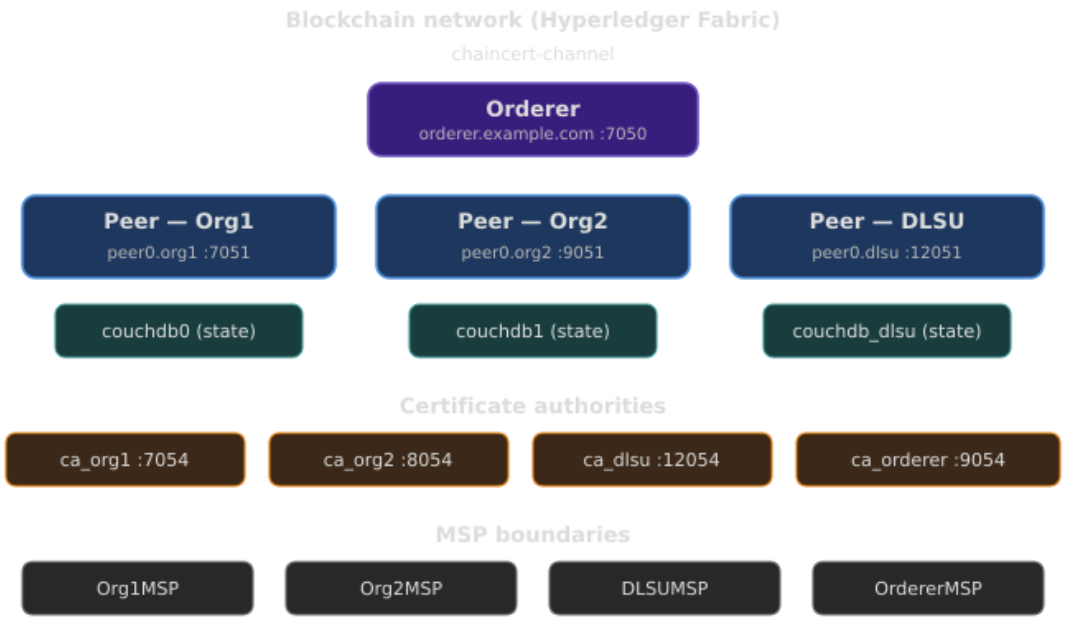


Figure 1: Blockchain network architecture showing all Fabric components

Orderer (1 node)

orderer.example.com (port 7050). Single Raft orderer. Every transaction that any peer submits goes to the orderer first. The orderer doesn't execute or validate the transaction — it decides the order in which transactions are placed into a block, batches them, and distributes those blocks to all peers. Analogy: a notary office that stamps sequence numbers and distributes copies.

Peer Nodes (3 nodes, 3 organizations)

peer0.org1 (:7051), peer0.org2 (:9051), peer0.dlsu (:12051). Each peer has two jobs: (1) execute chaincode when a transaction proposal comes in (endorsement), and (2) maintain their own copy of

the ledger and world state. Each peer's state DB (couchdb0, couchdb1, couchdb_dlsu) is a snapshot of current values so you don't replay the entire chain. Analogy: branch offices of a bank, each with their own ledger copy.

Certificate Authorities (4 CAs)

ca_org1 (:7054), ca_org2 (:8054), ca_dlsu (:12054), ca_orderer (:9054). Each CA issues X.509 certificates for its organization. These certificates are what make Fabric permissioned. Register = apply for a passport. Enroll = pick up the passport.

Why does the orderer have its own CA? The orderer is its own organization — it needs identity to authenticate itself to peers (TLS), sign blocks, and establish encrypted connections. Analogy: the postal service isn't a citizen of any country but still needs official ID so embassies can verify deliveries are genuine.

4.2 Full Application Architecture

The complete stack: React frontend → apiService.js (Axios + JWT) → Express backend (port 4000) → Routes → Controllers → CouchDB service + Fabric blockchain.

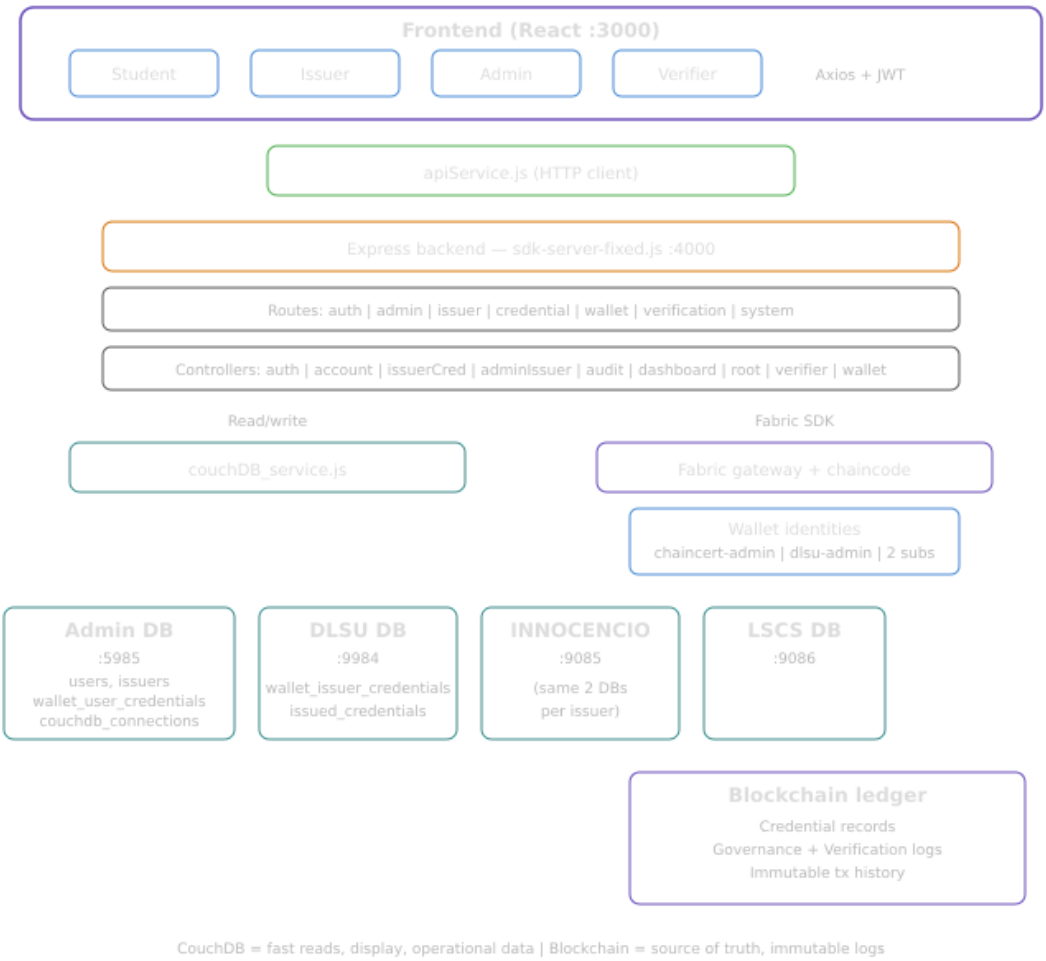


Figure 2: Full application architecture showing frontend, backend, CouchDB instances, and blockchain

4.3 Transaction Lifecycle

When a credential is issued, the transaction passes through: proposal (signed by wallet identity) → endorsement (peers execute chaincode independently) → ordering (orderer batches into block) → validation + commit (all peers append block to ledger). The result: 3 identical ledger copies = tamper-evident.

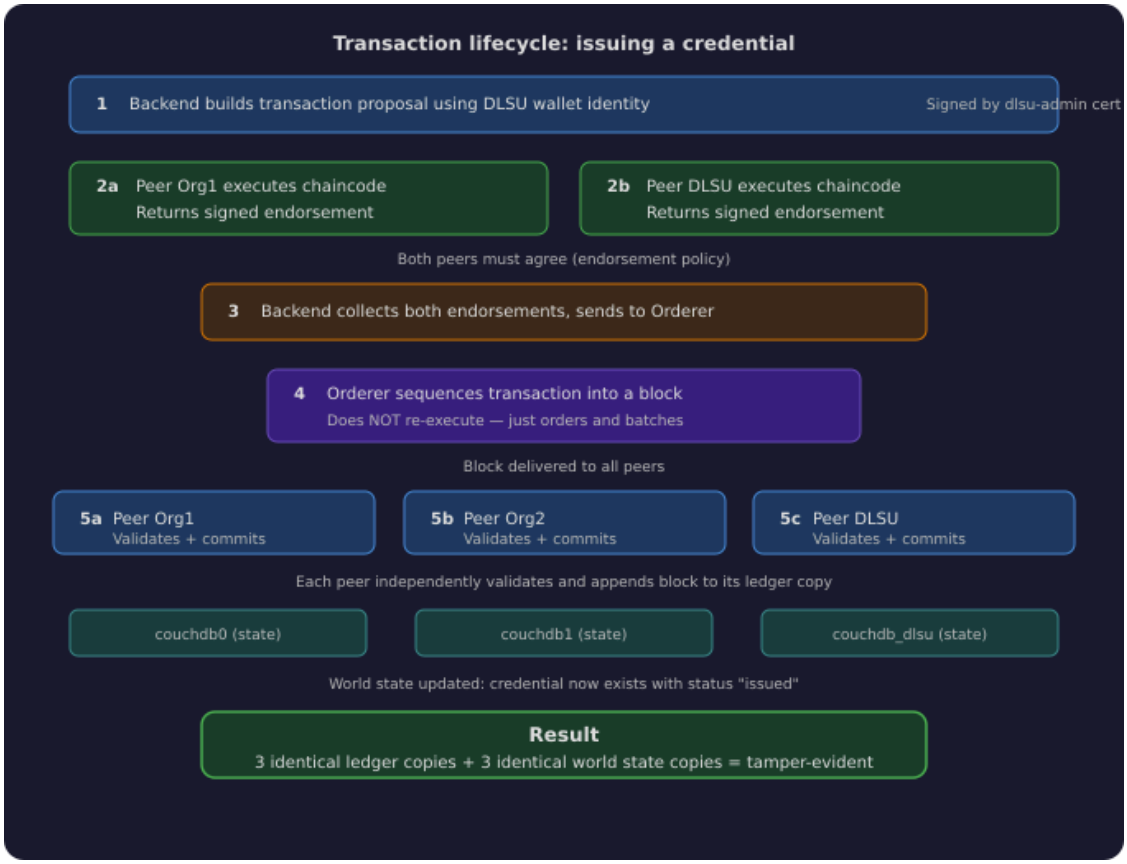


Figure 3: Transaction lifecycle for credential issuance through the Fabric network

4.4 Current vs Consortium Model

The current platform model has ChainCert controlling all infrastructure on one machine. The future consortium model distributes infrastructure across org-owned servers, with channel config updates requiring multi-org cryptographic signatures.



Figure 4: Current platform model vs future consortium model side-by-side

5. Hyperledger Fabric / Design Choices

❓ Why Hyperledger Fabric instead of a public blockchain?

Summary:

The system needs: permissioned access, real organizational identity, privacy controls, no gas fees, and enterprise-style governance. Public blockchain gives immutability and transparency but exposes data publicly and lacks org-level identity control. Fabric provides: MSP/CA identity model, permissioned access, privacy features, and enterprise control.

✅ **Defense Answer**

"We selected Hyperledger Fabric not just for immutability, but for permissioned participation, identity management, privacy controls, and enterprise suitability, all of which are essential for credential systems."

❓ What is the difference between the ChainCert network and chaincert-channel?

Summary:

They are related but not the same. The "network" is the entire Hyperledger Fabric infrastructure — all orderers, peers, CAs, MSP definitions, and channel configurations. A "channel" is a private ledger within that network — a subset of organizations sharing a specific ledger. chaincert-channel is a channel within the ChainCert Fabric network. One network can host multiple channels. Currently, ChainCert uses a single channel. In a future consortium model, the same network could host additional channels without requiring a separate network deployment.

Analogy: the network is the building, the channel is a meeting room inside that building. Everyone shares infrastructure (elevators, security, power), but different rooms have different participants and conversations.

✅ **Defense Answer**

"The ChainCert network is the full Fabric infrastructure — orderers, peers, CAs, and MSP configurations. chaincert-channel is a single channel within that network where all current organizations share one ledger. A Fabric network can host multiple channels, each with different participants and different ledger data."

❓ Who handles MSPs, CAs, and peer nodes? Does each issuer manage all of these?

Summary:

In Design 1 (current): ChainCert handles everything — all CAs, all peers, all MSP directories, the orderer. The issuer (e.g., DLSU) never touches infrastructure. They log in to the web dashboard and issue credentials. ChainCert created containers named after them (ca_dlsu, peer0.dlsu) and generated their MSP materials during the node creation process.

In Design 2 (hybrid): The issuer receives a deployment kit and runs their own ca, peer, and CouchDB on their own server. They generate their own MSP materials and send the MSP definition (root cert + admin cert) back to ChainCert for inclusion in the channel configuration. ChainCert still runs the orderer and base network.

In Design 3 (full consortium): Same as Design 2 infrastructure, but the issuer also participates in governance decisions (signing channel config updates).

The three components are connected as layers of the same identity system: CA issues the certificate → certificate goes into the MSP folder → MSP folder is loaded by the peer at startup → peer uses MSP rules to validate all incoming transactions. CA handles issuance, MSP handles policy, peer handles enforcement.

✅ Defense Answer

"In our current Design 1, ChainCert manages all infrastructure on behalf of issuers. Issuers interact only through the web interface. In Design 2, issuers deploy their own peer, CA, and CouchDB using a provided deployment kit while ChainCert retains governance. The CA, MSP, and peer are three layers of the same identity system — the CA issues certificates, the MSP defines which certificates are trusted, and the peer enforces those rules during transaction validation."

❓ Who is the orderer? Who are Org1, Org2, DLSU, and OrdererMSP?

Summary:

Four organizations exist in the ChainCert Fabric network:

Org1MSP (peer0.org1, port 7051, CA port 7054) = ChainCert platform itself. The chaincert-admin Fabric identity is derived from Org1's CA. When the backend performs system-level blockchain operations

(admin audit logs, verification logs, integrity checks), it uses chaincert-admin which is an Org1MSP identity.

Org2MSP (peer0.org2, port 9051, CA port 8054) = Second base organization from Fabric's test-network. Exists because test-network ships with two orgs by default and the endorsement policy requires multiple peers. Serves as a second endorsing peer — a built-in second witness for transaction validation.

DLSUMSP (peer0.dlsu, port 12051, CA port 12054) = First real issuer organization added to the network. Has its own MSP, peer, CA, and subordinate identities (INNOCENCIO, LSCS).

OrdererMSP (orderer.example.com, port 7050, CA port 9054) = The orderer organization. Its sole job: receive endorsed transactions, sequence them into blocks (every 2 seconds or 10 transactions per configtx.yaml BatchTimeout/MaxMessageCount), deliver blocks to all peers. Does not execute chaincode, does not store application data.

Analogy: Org1 = ChainCert headquarters. Org2 = a built-in auditor. DLSU = a client institution. The orderer = the court clerk who doesn't decide cases but files and distributes documents in the correct order.

> ✅ **Defense Answer**
>
> *"Org1 is ChainCert — the platform operator. Org2 is a second endorsing peer from Fabric's test-network that provides independent transaction validation. DLSU is the first real issuer organization added. The orderer is a separate organization that sequences transactions into blocks — it needs its own CA for TLS authentication and block signing. In our current deployment, ChainCert controls all components including the orderer."*

❓ What Fabric features are actually used in ChainCert and how?

Summary:

MSP (Membership Service Provider): Defines organizational identity boundaries. Org1MSP = ChainCert, DLSUMSP = DLSU and its subordinates. Every transaction carries the signer's MSP identity. Used in: sdk-server-fixed.js resolves Fabric identity by orgName, wallet stores identities with mspId, connection profiles map MSPs to peers.

Fabric CA: Issues X.509 certificates. Register = create identity entry in CA. Enroll = generate certificate + private key. These certificates become the .id files in the wallet/ directory. Used in: registerEnrollFORDLSU.sh, setupWallet_DLSU.js.

Channels: Private ledgers shared by subset of organizations. Currently one channel (chaincert-channel) with all orgs. Used in: gateway.getNetwork('chaincert-channel') in sdk-server-fixed.js.

Endorsement Policy: Defines which peers must independently execute and agree on a transaction. MAJORITY Endorsement — when DLSU issues a credential, multiple peers run the chaincode independently and must return the same result.

Immutable Transaction Log: Append-only blockchain. World state changes but transaction history cannot be altered. Used in: Get_CredentialTemplateOrIssuedCredentialHistory chaincode function reads from the log, powering the credential timeline UI.

Not currently used but available: Private Data Collections, distributed channel governance. These are future-use features for consortium deployment.

You are not underusing Fabric by not implementing full consortium governance — MSP, CA, channels, endorsement, and immutable logging are all actively used.

> ✅ **Defense Answer**
>
> *"We actively use Fabric's MSP identity model, Certificate Authorities, channel architecture, endorsement policy, and immutable transaction logging. These features provide permissioned access, organizational trust boundaries, independent transaction validation, and complete auditability. Private data collections and distributed governance are available Fabric features for future consortium deployment but are not required for the current platform model."*

> ❓ **Why not one node with multiple Fabric identities, since issuers don't own nodes anyway?**

Summary:

Technically possible but architecturally weak. A single node with many identities removes meaningful org boundaries, weakens endorsement logic, and turns the design into a multi-user system instead of a multi-organization system. The current multi-org structure preserves separate MSPs, logical trust domains, better future scalability, and more defensible Fabric architecture.

> ✅ **Defense Answer**
>
> *"We preserved multiple organizational identities and MSP boundaries because Hyperledger Fabric is built around trust domains, not just user identities. A single-node multi-identity model would weaken that architecture."*

> ❓ **Who operates the orderer in each deployment design? What is a Raft cluster?**

Summary:

The orderer is a shared network service that sequences transactions into blocks and delivers them to all peers. It is NOT a per-org component.

Design 1 (current): ChainCert runs the single orderer on the same machine.

Design 2 (hybrid): ChainCert still runs the orderer. Issuers send transactions to ChainCert's orderer for sequencing.

Design 3 (full consortium): The orderer SHOULD be distributed via Raft consensus.

What is Raft: A protocol for a group of nodes to agree on the order of entries in a shared log. One node is elected leader, proposes blocks, followers verify and acknowledge. Once majority acknowledges, block is committed. If leader crashes, new leader elected within seconds. If leader tries to exclude a transaction, followers reject.

In Fabric: Instead of one orderer container, there would be 3 or 5 operated by different organizations. All form one Raft cluster, communicate, elect leader, collectively produce blocks. Configured in configtx.yaml by listing multiple entries under EtcdRaft.Consenters.

Why not multiple orderers in Design 1: All would be on the same machine. Multiple orderer containers on one machine is like putting 3 copies of a key in the same drawer. Zero additional security. Raft does not change how transactions work from the application's perspective — it is purely an infrastructure-level change.

> ✅ **Defense Answer**
>
> *"In our current setup and in Design 2, ChainCert operates the single orderer — consistent with centralized governance. In Design 3, the orderer should be distributed using Raft consensus where multiple organizations each operate an orderer node that collectively agree on block ordering. We use a single orderer because multiple orderers only provide value when distributed across independent servers, which is not the case in our single-machine prototype."*

❓ What are channels, consortiums, MSPs, peers, orderers, and dev containers?

Summary:

MSP = org identity boundary. CA = certificate issuer. Peer = keeps ledger and executes smart contracts. Orderer = orders transactions into blocks. Channel = private ledger between selected orgs. Consortium

= group of orgs allowed to create/manage a channel. Dev containers = chaincode runtime containers. Formula: Number of dev containers = Number of Peers × Number of Chaincodes.

❓ What is the difference between MSP and CA? How do they relate?

Summary:

The CA (Certificate Authority) is a running service — a Docker container (e.g., ca_dlsu on port 12054) that registers identities and issues X.509 certificates. It is operational. It does things: register creates an entry in the CA database with a name and secret, enroll generates a signed certificate and private key for that identity.

The MSP (Membership Service Provider) is a configuration folder — not a running service. It contains the root CA certificate (so peers know which CA to trust), admin certificates (so peers know who has admin rights), and NodeOU rules (distinguishing admin/peer/client/orderer roles within the org).

Analogy: the CA is the machine that prints ID cards. The MSP is the security policy that says which ID cards are accepted at the door.

When a peer receives a transaction signed by "delasalleuniversity-admin," it checks: does this certificate chain back to a CA root certificate listed in DLSUMSP's trusted certificates? If yes, the identity is valid for DLSU operations. If someone presents a certificate issued by ca_org1 but claims to be DLSU, DLSUMSP will reject it because that CA root is not in DLSU's trust list.

Each organization has both: ca_dlsu issues DLSU certificates, DLSUMSP defines the trust boundary for DLSU identities. The CA creates identities; the MSP defines who is trusted; the peer enforces the rules.

Why separate CAs per org instead of one shared CA: if a single CA is compromised, every identity in the entire network is compromised. With separate CAs, compromising ca_dlsu only affects DLSU identities — Org1 and Org2 remain safe. In the current Design 1 prototype, all CAs are on the same machine so this separation is logical rather than physical. The real security benefit activates in Design 2/3 where each org's CA runs on their own server.

✅ Defense Answer

"The CA is the service that issues certificates — it creates identities. The MSP is the configuration that defines which certificates are trusted — it enforces organizational boundaries. Each organization has its own CA and MSP because Fabric is built around organizational trust domains. A certificate issued by DLSU's CA is only valid within DLSU's MSP boundary. Separate CAs ensure that compromising one organization's certificate authority does not compromise the entire network."

6. Issuer Onboarding / Operational Reality

❓ What is the practical issuer onboarding flow?

Summary:

Current flow: (1) issuer registers with documents, (2) if subordinate: root authority reviews and approves/rejects first (two-tier approval gate), (3) admin reviews, (4) if legit, ChainCert manually prepares blockchain-side setup (Fabric identity, wallet import, CouchDB container), (5) admin approves in system (assigns fabricIdentity, couchDBUrl, walletId), (6) approval/rejection logged on blockchain, (7) admin notification created, (8) issuer can use the platform.

Technical Detail:

The two-tier subordinate approval flow is a real governance feature: root authority must approve first (rootApprovalStatus → approved), then admin can approve. Admin is blocked from approving until root approves — enforced in the Approve_Issuer CouchDB service function.

✅ **Defense Answer**

"Issuer onboarding in the current implementation combines application-level registration with manually managed blockchain integration, including a two-tier approval gate for subordinate issuers, which is appropriate for a managed platform environment."

❓ Why is node deployment manual instead of automated?

Summary:

Four reasons:

First, automated Fabric node provisioning is a complex infrastructure problem that dedicated open-source projects (Hyperledger Bevel, Minifab) were built specifically to solve. Building our own automation would be a separate project.

Second, manual deployment is the universal process that works across all three designs. In Design 2 and 3, the issuer deploys on their own server using a deployment kit — ChainCert cannot remotely automate deployment on someone else's infrastructure.

Third, root authority onboarding is an infrequent, high-stakes operation requiring human judgment at every step. Automating something that happens once every few months would be premature optimization. Subordinate issuers already have a simpler process.

Fourth, the test-network itself is designed for manual org addition. network.sh automates Org1 and Org2 but explicitly does not support adding arbitrary orgs dynamically.

> ✅ **Defense Answer**
>
> *"Node deployment is manual because automated Fabric provisioning is a complex infrastructure problem outside capstone scope — dedicated tools like Hyperledger Bevel exist specifically for this. Root authority onboarding is also an infrequent operation requiring human judgment at every step. The manual process maps naturally to all three deployment designs."*

❓ Why are DLSU and subordinates the only fully working issuer accounts?

Summary:

Because only they have: actual blockchain identity/integration completed, Fabric CA registration and enrollment done, wallet identity imported, CouchDB containers running with databases created, and couchdb_connections entry registered.

> ✅ **Defense Answer**
>
> *"Only pre-configured issuers such as DLSU and its subordinates are fully operational because credential issuance requires completed blockchain identity and infrastructure setup, which is not automatically generated for all newly registered issuers."*

❓ How should UAT users test issuer registration and operations?

Summary:

The system has built-in safety checks that prevent issuer activation without proper infrastructure. When admin clicks approve, the Approve_Issuer function resolves the CouchDB URL from couchdb_connections by orgName. If no entry exists, the approval throws an error and fails.

Practical UAT approach:

Registration and onboarding flow (form, document upload, pending status, admin review): any tester with any unique email. Multiple testers can test simultaneously. Rejection works fully — re-registration is allowed after rejection.

Full approval testing: testers must register using an org name with a couchdb_connections entry — currently delasalleuniversity, innocencio, or lasallecomputersociety only. Other org names will cause the approve button to fail with "No CouchDB connection found." This is a safety feature.

Credential operations: testers must use pre-deployed accounts (dlsu@email.com, subordinate@email.com). For holder/verifier testing: kyle_justin_tan@dlsu.edu.ph and admin1@email.com / admin2@email.com.

> ✅ **Defense Answer**
> *"UAT testers can test the full registration and onboarding workflow with any email address. Full approval requires an org with pre-deployed blockchain infrastructure — the system actively blocks approval if no CouchDB connection mapping exists for the org. Credential operations are limited to pre-configured accounts because blockchain identity and infrastructure setup is a manual process. This built-in check is a safety feature that prevents issuer activation without proper infrastructure."*

❓ Why were these specific documents chosen as issuer requirements? What is the rationale per category?

Summary:

All five issuer categories follow a three-layer validation framework:

Layer 1 — "Are you real?" — Prove the organization legally exists. This is the SEC registration, CHED recognition, Republic Act, or DTI registration depending on the type of org. Without this, anyone could register as a fake institution and start issuing credentials through ChainCert.

Layer 2 — "Are you qualified to issue credentials?" — Prove the organization has the authority or competence to issue the specific type of credential they claim. A legally registered company does not automatically have the right to issue academic degrees. This is where category-specific requirements diverge.

Layer 3 — "Did your organization actually authorize this?" — Prove that the person registering has institutional permission to participate. This is always some form of board resolution or office order. Without this, a random employee could sign up on behalf of their entire institution without leadership knowing.

Category-specific rationale:

University — CHED recognition proves authority to grant degrees (not just legal existence). Legal personality document proves the institution itself exists as an entity. Board resolution proves leadership authorized ChainCert participation and named specific officials. Quality accreditation (optional) is a trust booster from independent review bodies like PAASCU.

Government — Legal creation instrument (Republic Act, Executive Order, local ordinance) proves the agency was formally created by law. Mandate document proves the agency has authority to issue the specific credential type. Office order (not board resolution — government agencies use orders, not corporate resolutions) proves leadership authorized participation. Sample credential template lets the admin verify the credential they claim to issue actually exists.

Training Provider — SEC/DTI registration proves business existence. Mayor's permit proves current operational status (renewed annually, unlike SEC which can be years old). TESDA CoPR proves the training program meets national standards and is not a random seminar. Program brief shows actual curriculum and assessment methods so the admin can verify substance.

Industry Certifier — SEC/charter proves legal existence. Industry/professional recognition proves the certification is actually respected by the relevant industry (without this, anyone could create a fake certification body). Documented certification scheme aligned with ISO/IEC 17024 principles proves the certification process is structured, repeatable, and governed — not arbitrary. Board resolution covers both scheme approval and ChainCert authorization.

Others — Minimal requirements (SEC/DTI + board resolution only). The admin reviews these with extra scrutiny since there is no sector-specific regulatory document to validate against.

The subordinate issuer additional requirement (Root Authority Authorization Letter) ensures the root authority explicitly vouches for the subordinate before the admin even reviews the application.

✅ **Defense Answer**

"Every issuer category follows three validation layers: legal existence, credential authority, and institutional authorization. The specific documents vary per category because different types of organizations are created, regulated, and governed through different mechanisms in the Philippines. For example, universities are recognized by CHED while training providers are registered with TESDA — each requires proof from their respective regulatory body. The document-based approach is stronger than a database lookup because a lookup only confirms an organization name exists in a list — it does not verify that the person registering is authorized to represent that organization. The board resolution or office order requirement closes that gap."

7. Security / Admin / Identity Model

This section covers what IS implemented first, then acknowledged trade-offs.

7.1 Implemented Security Measures

- JWT authentication with middleware on every request (sdk-server-fixed.js)
- bcrypt password hashing (bcryptjs, 10 rounds)
- Fabric X.509 certificates for all blockchain identities
- On-chain audit logging for all governance actions (approvals, rejections, status changes, verifications, password changes)
- Email validation regex on registration
- Same-password rejection on forgot-password reset
- Disabled account login blocking (both users and issuers)
- Disabled user issuance blocking (excluded from eligible users dropdown)
- CouchDB vs blockchain integrity check (field-by-field comparison)
- Two-tier subordinate approval (root authority gate before admin approval)
- Credential revocation notifications to holders
- Status transition guards (prevent re-approving already approved issuers, etc.)

7.2 Security Questions

> ❓ **Is Fabric identity tied to the admin account?**

Summary:

No. Fabric identities are tied to blockchain operations at the organizational level, not to individual admin accounts. Multiple admin accounts use the same ChainCert Fabric admin identity (chaincert-admin) when performing blockchain operations. Individual accountability is preserved through logged metadata (admin email) in both blockchain and off-chain records.

> ✅ **Defense Answer**
>
> *"Fabric identities are tied to blockchain operations at the organizational level rather than to individual admin accounts. In our system, admins act as operators of the ChainCert platform, so they share a single operational Fabric identity. Individual accountability is still preserved through logged metadata such as admin email in both blockchain and off-chain records."*

> ❓ **How do you know which admin performed which action?**

Summary:

Even though the Fabric identity is shared, individual accountability is preserved through logging. The system tracks admin email, action performed, and timestamp — stored in blockchain transaction metadata and CouchDB audit logs. Blockchain shows which organization performed the action; logs show which admin account triggered it.

> ✅ **Defense Answer**
>
> *"Although multiple admin accounts share the same operational Fabric identity, we preserve accountability by explicitly logging the admin account responsible for each action. This allows us to combine blockchain immutability with application-level attribution."*

7.3 Acknowledged Limitations

- Shared Fabric admin identity (no per-admin cryptographic attribution)
- Admin compromise risk (malicious approvals possible, but traceable)
- Mutable CouchDB read layer (mitigated by blockchain verification + integrity check)
- Onboarding legitimacy depends on admin validation ("garbage in" risk)
- Governance is still centralized

> ✅ **Defense Answer**
>
> *"The system significantly improves integrity, auditability, and verification through blockchain, but we acknowledge that governance and onboarding still rely on trusted administrative controls. These are known trade-offs of a platform-managed permissioned model and can be strengthened in future iterations through per-admin identities and more distributed approval mechanisms."*

8. Feature Inventory

Compact reference of every major feature implemented in ChainCert.

Feature	Description
Credential template creation	Blockchain + CouchDB dual write with skills, badge image, metadata
Credential issuance	Blockchain + dual CouchDB write with holderName enrichment, templateId tracking, duplicate prevention
Credential revocation	Blockchain + dual CouchDB sync + holder notification with optional rejection reason
Credential expiry automation	Midnight cron job + manual admin trigger, CouchDB first then blockchain logging
Credential verification	CouchDB + blockchain cross-check, on-chain verification log created
Duplicate issuance prevention	Blocks re-issuing same template to same holder with active/non-expired credential
Admin notifications	Triggered on issuer registration, root subordinate decisions, admin approval/rejection/disable
User notifications	Triggered on credential issuance and revocation
Profile sharing (public)	No-auth share links with view counting, expiry, and revocation
Profile sharing (OTP)	Recipient-specific links requiring email + 6-digit OTP verification
Visibility toggle	Holders: public/private per credential. Issuers: public/private per template
Two-tier subordinate approval	Root authority approves first, then admin can approve. Admin blocked until root gate passes
Rejected issuer re-registration	Old rejected doc deleted, fresh application proceeds
CouchDB vs blockchain integrity check	Field-by-field comparison across all issued credentials per issuer
PDF export	Single credential + portfolio export with profile links, numbered pages, holder info
Forgot password	OTP via email, same-password rejection, time-limited codes
Admin password change	Current password re-entry required, audit logged on-chain
Issuer document management	Files stored as CouchDB attachments, viewable/downloadable by admin
Partial success handling	If blockchain succeeds but CouchDB fails, returns warning instead of failing entirely

9. Data Design / CouchDB

> ❓ **Do you need controlled fields/schema in CouchDB?**

Summary:

Yes — CouchDB is schemaless, which means any field can be added or missing. Currently, schema enforcement happens in the controller layer (validation before writes) rather than at the database level. A formal schema validation middleware layer would be a recommended improvement.

10. Implementation / Code Flow

> ❓ **What is the general operation/data flow?**

Summary:

Frontend file → apiService → backend route file → sdk-server-fixed.js → controller file uses blockchain (index.js chaincode) AND/OR CouchDB (couchDB_service.js) → response goes back up the same path.

Where should a feature fetch data from? Current/world state = CouchDB. Any LOGS or audit = blockchain. Rule of thumb: display-heavy/dashboard/list view → CouchDB. Final validation/immutable check/trust-critical → blockchain.

Example Flow: Issue Credential

- Issuer dashboard calls apiService.Issue_Credential
- apiService calls POST /credentials/:credentialId/issue
- credentialRoutes.js routes to issuerCredentialController.Issue_Credential
- Controller resolves issuer Fabric identity from CouchDB
- Fabric SDK submits IssueCredential to blockchain
- Controller enriches credential with holderName, badgeImage, skills from template
- Issue_Credential_DB saves to issuer CouchDB (issued_credentials) and admin CouchDB (wallet_user_credentials)
- Create_CredentialNotification sends notification to holder

Example Flow: Revoke Credential

- Controller submits RevokeCredential to blockchain with rejection reason
- Revoke_Credential_DB updates both CouchDB copies (issuer + admin wallet)
- Revocation notification sent to holder

Example Flow: Verify Credential

- Verifier inputs credential ID on dashboard
- Backend checks CouchDB (Verify_Credential_DB) for fast lookup
- Blockchain VerifyCredential chaincode executed for authoritative check
- LogVerification creates on-chain verification audit log

11. Deployment / Network Notes

❓ What happens when the network is brought down/up after adding orgs?

Summary:

When brought down, ALL containers including DLSU are removed. Added orgs like DLSU are NOT automatically restored by network.sh up — only Org1 and Org2 are hardcoded in the base test network script. Extra orgs must be re-created via node creation steps.

❓ Does deploying chaincode affect DLSU automatically?

Summary:

No. Base deployCC script only covers Org1 and Org2. DLSU needs separate manual chaincode install/approve/deploy steps (in node creation documentation).

❓ What do Docker volumes contain and does deleting them wipe records?

Summary:

Yes — deleting relevant Docker volumes wipes blockchain ledger state, CouchDB data, peer state, and CA data. Useful for resetting demo environment before defense.

✅ **Defense Answer**

"For a fresh demo environment, clearing the relevant Docker volumes resets both ledger and database state, allowing the team to demonstrate the full credential lifecycle from a clean start."

12. Miscellaneous Design Decisions

❓ Username/profile edits — should old records change?

Summary:

No. Profile updates are current-state updates, not historical rewrites. CouchDB user profiles update, but historical records (logs, issued credentials, verification records) preserve a snapshot of data at the time of the event.

✅ **Defense Answer**

"Profile edits update the user's current profile document, but they do not retroactively modify historical records. Event logs and audit records preserve a snapshot of data at the time of execution to maintain integrity and traceability."

❓ What if an organization wants its own independent ChainCert network?

Summary:

It depends on what "full control" means:

If they want private data but shared verification: a separate private channel within the same network is sufficient. Full credential records on their private channel, hashed/redacted versions on the shared chaincert-channel for cross-institutional verification.

If they want complete independence — their own admin, governance, orderer, no shared ledger: they need a completely separate Fabric network. Their credentials would not be verifiable through ChainCert's interface unless a cross-network bridge is built.

The critical question: does the issuer still want shared verification? If yes, same network with private channel. If no, separate network entirely. Separate channel = data privacy within shared trust. Separate network = full sovereignty but breaks shared verification.

✅ **Defense Answer**

"If an institution wants data privacy while remaining part of the ChainCert verification ecosystem, a private channel within the same network is the correct approach. If they want full sovereignty with no shared ledger, that requires a separate network deployment, which means credentials would not be verifiable through ChainCert unless a cross-network integration is built."

❓ Should credential templates be editable after creation?

Summary:

No. Once a template has been used to issue credentials, editing it would create inconsistency — existing credentials on the blockchain reference the original template data. The blockchain record is immutable, so issued credentials would mismatch the edited template. The correct approach: if a new version is needed, create a new template. This is standard practice in credential systems (Credly works the same way). Immutability is a feature, not a limitation.

✅ Defense Answer

"Credential templates are intentionally immutable after creation. Editing a template that has already been used to issue credentials would create inconsistency between the blockchain record and the template definition. The correct approach is to create a new template version, which preserves the integrity of all previously issued credentials."

13. Hostile Panelist Questions — High Risk

Questions most likely to cause difficulty if unprepared. Each includes the attack angle and a prepared defense.

> ⚠️ **Your JWT secret has a hardcoded fallback of 'dev_secret_key'. What happens if .env is missing?**

sdk-server-fixed.js line 17: process.env.JWT_SECRET || 'dev_secret_key'. If .env isn't loaded, anyone who knows that string can forge any JWT token. This is a prototype configuration — in production, the fallback would be removed and the server would refuse to start without a proper secret.

> ✅ **Defense Answer**
>
> *"The hardcoded fallback is a development convenience that would be removed in production deployment. In production, the server should fail to start if JWT_SECRET is not properly configured. We acknowledge this as a prototype-specific configuration choice."*

> ⚠️ **Your CouchDB credentials are hardcoded as admin:adminpw across every file. How do you manage database security?**

All CouchDB connection strings use hardcoded credentials. In the current prototype, all services run on localhost making external access unlikely. In production, credentials would be moved to environment variables and CouchDB instances would be network-isolated.

> ✅ **Defense Answer**
>
> *"The hardcoded credentials are appropriate for our isolated development environment where all services run on localhost. In a production deployment, credentials would be managed through environment variables and database instances would be network-isolated with proper access controls."*

> ⚠️ **What prevents a rogue admin from creating a fake issuer, approving it, and issuing fraudulent credentials?**

Nothing prevents it at the governance level — admin has full control. However: (1) every approval is logged on blockchain immutably, (2) every credential issuance is recorded, (3) the integrity check can detect mismatches, (4) fraudulent credentials can be investigated and revoked. The answer is detection and accountability, not prevention.

> ✅ **Defense Answer**
>
> *"The administrator remains a trusted role in our platform model. However, unlike traditional systems, every governance action is immutably recorded on blockchain. A rogue admin's actions would be permanently traceable, detectable through audit logs, and correctable through credential revocation. The system provides accountability rather than prevention."*

⚠️ You store JWT tokens in localStorage. That's vulnerable to XSS. Why not httpOnly cookies?

axiosInstance.js reads from localStorage. Any XSS vulnerability would expose the token. httpOnly cookies would prevent JavaScript access to tokens. This is a known web security trade-off chosen for development simplicity. The frontend uses React which provides JSX-level XSS protection.

> ✅ **Defense Answer**
>
> *"We acknowledge that localStorage-based token storage is less secure than httpOnly cookies against XSS attacks. React's built-in JSX escaping provides baseline XSS protection, and migrating to httpOnly cookies is a recommended production hardening step."*

⚠️ You have no rate limiting on any endpoint. What stops brute-force attacks?

No rate limiter middleware exists in sdk-server-fixed.js. Express doesn't rate-limit by default. In production, express-rate-limit or similar middleware would be added to login, registration, OTP verification, and verification endpoints.

> ✅ **Defense Answer**
>
> *"Rate limiting is not implemented in the current prototype. In production, we would add rate-limiting middleware on authentication, OTP, and public verification endpoints to prevent brute-force attacks."*

⚠️ Everything runs on HTTP, not HTTPS. JWT tokens and passwords travel in plaintext.

The system uses http://localhost for both frontend and backend. No TLS anywhere. This is acceptable for a development/prototype environment where all traffic is local. Production deployment would require TLS termination.

> ✅ **Defense Answer**
>
> *"The current prototype runs entirely on localhost where traffic does not traverse public networks. Production deployment would require HTTPS with proper TLS certificates for all communication."*

> ⚠️ **You have a single orderer node. If it goes down, the entire blockchain network stops. How is that not worse than what you're solving?**

The single orderer is a test-network limitation, not an architectural choice. Fabric supports Raft consensus with 3 or 5 orderers for fault tolerance. The system's value proposition is credential integrity and verification, not high-availability infrastructure.

> ✅ **Defense Answer**
>
> *"The single orderer is a limitation of the Hyperledger Fabric test-network used for prototyping. Production deployment would use a Raft consensus cluster with multiple orderer nodes for fault tolerance. The system's primary contribution is credential integrity and verification, which remains valid regardless of infrastructure redundancy."*

> ⚠️ **What happens to issued credentials if an issuer is disabled?**

Verification still works because the credential history and verification endpoints use chaincert-admin identity (not the disabled issuer's identity). Student dashboard still shows credentials because wallet_user_credentials is in the admin CouchDB, independent of the issuer's CouchDB.

> ✅ **Defense Answer**
>
> *"Disabling an issuer does not affect existing credentials. Verification continues to work through the system admin identity, and holders retain access to their credentials through the admin-side wallet database. Only new issuance operations from that issuer are blocked."*

14. Hostile Panelist Questions — Design-Level Attacks

⚠️ Your dual-write pattern returns 201 with success:false when CouchDB fails. Does the frontend treat that as success?

Create_CredentialTemplate returns HTTP 201 but JSON success:false on partial failure. If frontend code checks response.status instead of response.data.success, it would misinterpret this. The credential exists on blockchain but not in CouchDB — the issuer dashboard won't show it, and there's no automatic retry mechanism.

✅ Defense Answer

"This is a known edge case in our dual-write architecture. The partial success response is designed to communicate that blockchain write succeeded but CouchDB synchronization failed. The frontend checks the success field in the response body, not just the HTTP status code. The manual integrity check can detect these orphaned records."

⚠️ Your Issue_Credential flow does 6 sequential DB calls after blockchain. What if it fails halfway?

There's no transaction wrapping the CouchDB operations. If issuer DB save succeeds but admin wallet save fails, the credential exists in blockchain + issuer DB but not the student wallet. The student won't see it. The only reconciliation mechanism is the manual integrity check.

✅ Defense Answer

"CouchDB does not support multi-database transactions, so sequential writes are the practical approach. Partial failures are handled by returning warnings to the frontend. The integrity check feature provides a reconciliation mechanism for detecting and investigating such edge cases."

⚠️ Your integrity check only scans from CouchDB to blockchain. What about blockchain records missing from CouchDB?

getAllIssuedCredentialsForIntegrity starts from CouchDB docs and looks each up on blockchain. If a credential was written to blockchain but the CouchDB write failed entirely, it would never appear in the check. This is a one-directional scan limitation.

✅ **Defense Answer**

"The current integrity check is designed as a CouchDB-to-blockchain validation. A reverse scan (blockchain-to-CouchDB) would require iterating all blockchain keys, which is operationally expensive. The one-directional check catches the most common scenario: CouchDB drift from blockchain truth."

⚠️ **Your VerifyCredential chaincode checks the status field, not the expiryDate directly. Expired credentials pass verification until the cron runs.**

Between the actual expiry moment and the next cron run (or manual trigger), the credential's blockchain status remains "issued" even though the date has passed. A verifier checking during this window gets "valid."

✅ **Defense Answer**

"The system processes expirations through scheduled checks rather than real-time date comparison at the chaincode level. This creates a brief window where newly expired credentials may still show as valid. The admin can trigger an immediate expiry check if needed. Adding real-time expiry date checking in the VerifyCredential chaincode is a valid enhancement."

⚠️ **Disabled users' existing JWT tokens remain valid for up to 24 hours after being disabled.**

JWT tokens have expiresIn: '24h' and there's no token blacklist. The middleware only verifies signature, not active status. A disabled user with a valid token can continue using the system until expiry.

✅ **Defense Answer**

"This is a known trade-off of stateless JWT authentication. Implementing a token blacklist or reducing token lifetime are valid improvements. The current 24-hour window balances usability with security for the prototype."

⚠️ **Your eligible users endpoint depends on CouchDB for duplicate prevention. If CouchDB write failed, a duplicate could be issued on blockchain.**

getBlockedHolderIDsByTemplate queries wallet_user_credentials. If the CouchDB copy failed during a previous issuance (partial success), the holder appears eligible again. Blockchain has no built-in duplicate check for templateId + holderID.

> ✅ **Defense Answer**
> *"Duplicate prevention relies on the CouchDB operational layer. In the rare case of a partial write failure, the integrity check would detect the orphaned blockchain record. Adding a blockchain-level duplicate check in the chaincode would provide stronger guarantees."*

> ⚠️ **No notification is created when a credential expires. The holder doesn't know their credential is invalid.**

checkAndExpireCredentials updates status in both DBs but never calls Create_CredentialNotification. The holder silently loses credential validity. This is a gap in the notification system.

> ✅ **Defense Answer**
> *"This is a valid observation. The expiry notification was not implemented in the current version. The holder can still see the updated status on their dashboard, but an explicit notification would improve the user experience. This is a recommended enhancement."*

15. Hostile Panelist Questions — Architectural Attacks

⚠️ You claim blockchain provides immutability. But RevokeCredential and ExpireCredential modify the world state. How is that immutable?

This tests the distinction between world state and transaction log. World state is mutable (current state of each key). The transaction log (ledger) is immutable — every change is recorded as a new transaction. The history API reads from the log, not just current state. So "active → issued → revoked" is a sequence of immutable log entries.

✅ Defense Answer

"The world state reflects the current status of a credential, which naturally changes through its lifecycle. What remains immutable is the transaction log — every status change (creation, issuance, revocation, expiration) is permanently recorded as a separate blockchain transaction. Our history API retrieves this complete audit trail, proving the full chain of events cannot be altered."

⚠️ If CouchDB is just a cache and blockchain is source of truth, why can't you rebuild CouchDB from blockchain?

Because CouchDB stores data that blockchain doesn't: user accounts, passwords, profile pictures, uploaded documents, notifications, share links, display preferences, badge images, holderNames, and issuer onboarding details. Blockchain only stores credential lifecycle records and governance logs. Losing CouchDB means losing the application layer.

✅ Defense Answer

"CouchDB is not a simple cache of blockchain data. It stores application-layer data that has no blockchain equivalent — user accounts, profile information, uploaded documents, notifications, and operational metadata. The correct framing is: blockchain stores integrity-critical credential records, CouchDB stores the broader application state."

⚠️ What if the holder edits the exported PDF and presents it as legitimate?

This is actually the system's strongest answer. PDFs are presentation-layer artifacts. Any edited PDF still cannot change the blockchain record. The Verifier Dashboard exists precisely for this scenario — verification always goes through the blockchain-backed endpoint, not the PDF.

> ✅ **Defense Answer**
>
> *"This is exactly why the Verifier Dashboard exists. PDFs, no matter how convincing, are never the source of truth. Any employer or institution that needs certainty uses the verification interface, which checks directly against blockchain records. An edited PDF would immediately fail official verification."*

⚠️ Your credential IDs use Date.now() plus a short random suffix. Isn't that predictable?

cred_${Date.now()}_${Math.random().toString(36).substr(2, 9)} — timestamp is guessable and Math.random() is not cryptographically secure. However: uniqueness is guaranteed by the blockchain (duplicate check exists via Check_CredentialExists), and prediction alone doesn't grant issuance authority. Only an authenticated issuer with a valid Fabric identity can write to the blockchain.

> ✅ **Defense Answer**
>
> *"Credential ID predictability does not create a security vulnerability because writing to the blockchain requires a valid Fabric identity and proper endorsement from multiple peers. The ID serves as a unique key, not a security token. Knowing the format cannot grant unauthorized issuance."*

⚠️ Your actual throughput — how many credentials can you issue per second?

Honest answer: not formally benchmarked. Each issuance involves: blockchain transaction (endorsement from 2 peers + ordering + commit), 2 CouchDB writes, 1 notification write, and holderName enrichment. Fabric test-network typically handles tens of transactions per second. The bottleneck is the sequential CouchDB writes after blockchain, not the blockchain itself.

> ✅ **Defense Answer**
>
> *"We have not performed formal throughput benchmarking as this is a prototype. Hyperledger Fabric test-networks typically support tens of transactions per second. The system's architecture separates blockchain writes from CouchDB writes, so display-layer operations do not impact blockchain throughput."*

Final Practical Reminders

🔹 Strongest Core Position

The current setup is correct, spec-compliant, and the best fit because ChainCert operates as a managed credential issuance and verification platform, not a governance-heavy consortium.

🔹 Most Important Distinction

You did not fully decentralize governance, but you successfully decentralized trust in credential data through blockchain immutability and verification.

🔹 Key Architectural Insight

The system supports multiple deployment models: platform (current), hybrid (node ownership without governance), consortium (full distributed governance).

Node ownership and governance authority are SEPARATE configurations in Fabric. The configtx.yaml Admins policy is the single config point that distinguishes Design 1, 2, and 3.

The difference lies in ledger sharing and control over infrastructure/governance.

🔹 Performance vs Trust

CouchDB = performance / read layer.

Blockchain = trust / validation layer.

Integrity check = bridge between the two.

🔹 Biggest Honest Limitations

Admin still controls onboarding.

Blockchain does not prevent bad input ("garbage in").

System is not fully trustless.

Advanced Fabric governance/privacy features are not fully utilized.

No rate limiting, no HTTPS, no token revocation in prototype.

Single orderer node = single point of failure.

All infrastructure on one machine = no genuine redundancy.

🔹 Best Final Defense Line

Our system intentionally uses a centrally managed permissioned blockchain model because it best fits a credential issuance and verification platform. ChainCert handles onboarding and infrastructure as a service provider, while blockchain is used to decentralize trust in credential data through immutability, auditability, and independent verification. More distributed governance and deployment models remain valid future extensions, but are not the most appropriate default for the present system.